

I N D E X

18F24CS026

Name Aditya kumar Singh Std _____ Sec _____

Roll No. _____ Subject _____ School/College _____

SL No.	Date	Title	Page No.	Teacher Sign / Remarks
1	10/03/26	Implementing FCFS	1	<u>Rm</u> 10/3/26
2	10/03/26	Implementing SJF	4	} <u>Rm</u> 17/3/26
3	10/03/26	Implementing Priority scheduling	10	
4	17/03/26	Implementing Round Robin	16	<u>Rm</u> 17/3/26
5	24/03/26	Simulating multi-level queue scheduling	22	<u>Rm</u> 24/3/26
6	31/03/26	EDF, RMS	30	<u>Rm</u> 31/3/26 (10)
7	21/04/26	Producer-consumer Philosophers - Dining	(10)	} <u>Rm</u> 28/4/26
8	28/04/26	Banker's Algorithm	<u>Rm</u> 28/4/26	
9	5/5/26	Request resource Algo.	(10)	} <u>Rm</u> .
10	5/5/26	Deadlock detection	(10)	
11	12/5/26	First, Best Worst Fit	(10)	
12	26/05/26	Page replacement	(10)	<u>Rm</u>



Program 1 $\Rightarrow$ Implementing FCFS

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
struct Process {
```

```
    int pid, at, bt, ct, tot, wt, remaining_bt;
```

```
};
```

```
float fcfs (struct Process PC[], int n) {
```

```
    int time = 0;
```

```
    float total_wt = 0, total_tat = 0;
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (PC[i].at > PC[j].at) {
```

```
                struct Process temp = PC[i];
```

```
                PC[i] = PC[j];
```

```
                PC[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (time < PC[i].at)
```

```
            time = PC[i].at;
```

```
            PC[i].ct = time + PC[i].bt;
```

```
            PC[i].tat = PC[i].ct - PC[i].at;
```

```
            PC[i].wt = PC[i].tat - PC[i].bt;
```

```
            total_wt += PC[i].wt;
```

```
            total_tat += PC[i].tat;
```

```
            time = PC[i].ct;
```

```
    }
```



```
int main () {
```

3

```
for (int i=0; i<n; i++) {  
    p1[i] = original[i];  
    p2[i] = original[i];  
    p3[i] = original[i];  
}
```

return 0;

Output :

Enter number of processes: 4

Enter Arrival Time and Burst Time for P1: 0 7

Enter Arrival Time and Burst Time for P2: 8 3

Enter Arrival Time and Burst Time for P3: 3 9

Enter Arrival Time and Burst Time for P4: 5 6

FCFS

Process	AT	BT	CT	TAT	WT
1	0	7	7	7	0
3	3	4	11	8	4
4	5	6	17	12	6
2	8	3	20	12	9

Average WT = 4.75

Average TAT = 9.75

10/3/25

SJF (Non Preemptive)

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid, at, bt, ct, tat, wt;
```

```
    int completed;
```

```
};
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```
    int time = 0, done = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].pid = i+1;
```

```
        printf("In Process %d\n", p[i].pid);
```

```
        printf("In Arrival Time: ");
```

```
        scanf("%d", &p[i].at);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &p[i].bt);
```

```
        p[i].completed = 0;
```

```
    }
```

```
    while (done < n) {
```

```
        int min = INT_MAX, index = -1;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (p[i].at <= time && p[i].completed == 0) {
```

```
                if (p[i].bt < min) {
```

```
                    min = p[i].bt;
```

```
                    index = i;
```

```
            }
```

```
        }
```

```
    }
```

```
        if (index != -1) {
```

```
            time += p[index].bt;
```

```
            p[index].ct = time;
```

```
            p[index].completed = 1;
```

```
            done++;
```

```
        } else {
```

```
            time++;
```

```
        }
```

```
    }
```

```
    double tatsum = 0;
```

```
    double wtsum = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].tat = p[i].ct - p[i].at;
```

```
        p[i].wt = p[i].tat - p[i].bt;
```

```
        tatsum += p[i].tat;
```

```
        wtsum += p[i].wt;
```

```
    }
```

```
    double tatavg = tatsum/n;
```

```
    double wtavg = wtsum/n;
```

```
    printf("In PID\t AT\t BT\t CT\t TAT\t WT\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d\t %d\t %d\t %d\t %d\t %d\n",
```

```
            p[i].pid, p[i].at, p[i].bt, p[i].ct,
```

```
            p[i].tat, p[i].wt);
```

```
    }
```

```
    printf("In Average TAT: %.2f", tatavg);
```

```
    printf("In Average WT: %.2f", wtavg);
```

```
    return 0;
```

```
}
```


Output:

Enter number of Process: 4

Process 1

Arrival Time: 1

Burst Time: 2

Process 2

Arrival Time: 2

Burst Time: 3

Process 3

Arrival Time: 1

Burst Time: 3

Process 4

Arrival Time: 4

Burst Time: 8

PID	AT	BT	CT	TAT	WT
1	1	2	3	2	0
2	2	3	6	4	1
3	1	3	9	8	5
4	4	3	12	8	5

Average TAT: 5.500

Average WT: 2.75

SJF (Preemptive)

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
struct Process {
```

```
    int pid, at, bt, ct, wt, tat, xt;
```

```
};
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of process: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```
    int time = 0, complete = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].pid = i + 1;
```

```
        printf("Process %d\n", p[i].pid);
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &p[i].at);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &p[i].bt);
```

```
        p[i].completed = 0;
```

```
    }
```

```
    while (done < n) {
```

```
        int min = INT_MAX, index = -1;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (p[i].at <= time && p[i].completed == 0) {
```

```
                if (p[i].bt < min) {
```

```
                    min = p[i].bt;
```

```
                    index = i;
```

```
                }
```

```
            }
```

```
    }
```



```

if (index != -1) {
    time += p[index].bt;
    p[index].ct = time;
    p[index].completed = 1;
    done++;
} else {
    time++;
}
}

double tatsum = 0;
double wtsum = 0;
for (int i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    tatsum += p[i].tat;
    wtsum += p[i].wt;
}

double tatavg = tatsum / n;
double wtagv = wtsum / n;
printf("m PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
           p[i].pid, p[i].at, p[i].bt,
           p[i].ct, p[i].tat, p[i].wt);
}

printf("m Average TAT: %f", tatavg);
printf("m Average WT: %f", wtagv);

return 0;
}
    
```

Output:

Enter Number of process: 4

Process 1

Arrival Time: 1

Burst Time: 2

Process 2

Arrival Time: 2

Burst Time: 3

Process 3

Arrival Time: 1

Burst Time: 3

Process 4

Arrival Time: 4

Burst Time: 1

PID	AT	BT	CT	TAT	WT
1	1	2	3	2	0
2	2	3	6	4	1
3	1	3	4	8	5
4	4	1	12	8	5

Average TAT: 5.5 ms

Average WT: 2.75 ms

Priority Scheduling (Non-Premptive)

```
#include <stdio.h>
```

```
struct Process {
```

```
int pid, bt, priority, wt, tat;
```

```
};
```

```
int main() {
```

```
int n, i, j;
```

```
struct Process p[10], temp;
```

```
float total_wt = 0, total_tat = 0;
```

```
printf("Enter number of process: ");
```

```
scanf("%d", &n);
```

```
for(i=0; i<n; i++){
```

```
printf("In Process %d\n", i+1);
```

```
p[i].pid = i+1;
```

```
printf("Burst Time: ");
```

```
scanf("%d", &p[i].bt);
```

```
printf("Priority: ");
```

```
scanf("%d", &p[i].priority);
```

```
};
```

```
for(i=0; i<n-1; i++){
```

```
for(j=i+1; j<n; j++){
```

```
if(p[i].priority > p[j].priority){
```

```
temp = p[i];
```

```
p[i] = p[j];
```

```
p[j] = temp;
```

```
};
```

```
};
```

```
{
```

```
p[0].wt = 0;
```

```
for(i=1; i<n; i++){
```

```
p[i].wt = p[i-1].wt + p[i-1].bt;
```

```
};
```

```
for(i=0; i<n; i++){
```

```
p[i].tat = p[i].wt + p[i].bt;
```

```
total_wt += p[i].wt;
```

```
total_tat += p[i].tat;
```

```
};
```

```
printf("In PID BT WT PR Tat\n");
```

```
for(i=0; i<n; i++){
```

```
printf("%d\t%d\t%d\t%d\t%d\n",
```

```
p[i].pid, p[i].bt, p[i].priority, p[i].wt,

```

```
p[i].tat);
```

```
};
```

```
printf("In Average Waiting time = %2f", total_wt/n);
```

```
printf("In Average Turnaround Time = %2f\n", total_tat/n);
```

```
return 0;
```

```
};
```


Output:

Enter Number of Process: 4

Process 1
Arrival Time: 1
Burst Time: 2

Priority: 3

Process 2

Arrival Time: 0

Burst Time: 3

Priority: 2

Process 3

Arrival Time: 2

Burst Time: 2

Priority: 1

Process 4

Arrival Time: 3

Burst Time: 2

Priority: 4

PID	AT	BT	PR	WT	TAT	CT
2	0	3	2	0	3	3
1	1	2	3	2	4	5
3	2	2	1	3	5	7
4	3	2	4	4	6	9

Gantt Chart:

IP2 IP1 IP3 IP4
0 3 5 7 9

Average waiting Time: 2.5ms

Average Turnaround Time: 4.5ms

* Priority Scheduling (Preemptive):

```
#include <stdio.h>
```

```
int main()
```

```
{ int n, i, time = 0, completed = 0;
```

```
int bt[10], wt[10], priority[10];
```

```
int min_poi, highest;
```

```
float total_wt = 0, ftotal_tat = 0;
```

```
printf("Enter number of processes: ");
```

```
scanf("%d", &n);
```

```
for (i = 0; i < n; i++)
```

```
{ scanf("%d", &n);
```

```
for (i = 0; i < n; i++)
```

```
{ printf("Process %d", i+1);
```

```
printf("Burst Time: ");
```

```
scanf("%d", &bt[i]);
```

```
printf("Priority: ");
```

```
scanf("%d", &priority[i]);
```

```
wt[i] = bt[i];
```

```
}
```

```
while (completed != n)
```

```
{ min_poi = 9999;
```

```
highest = -1;
```

```
for (i = 0; i < n; i++)
```

```
{ if (wt[i] > 0 && priority[i] < min_poi)
```

```
{ min_poi = priority[i];
```

```
highest = i;
```



```

    bt[highest]--;
    time++;
    if (bt[highest] == 0) {
        completed++;
        tat[highest] = time;
        wt[highest] = tat[highest] - bt[highest];

        total_wt += wt[highest];
        total_tat += tat[highest];
    }

    printf("\n Process ID BT PR WT TAT\n");

    for (i = 0; i < n; i++) {
        printf("P: %d BT: %d PR: %d WT: %d TAT: %d\n",
            i+1, bt[i], priority[i], wt[i], tat[i]);
    }

    printf("\n In Average waiting Time = %2f", total_wt/n);

    printf("\n In Average Turnaround Time = %2f", total_tat/n);

```

return 0;

Output:

Enter number of process: 4

Process 1

Arrival Time: 1

Burst Time: 2

Priority: 3

Process 2

Arrival Time: 0

Burst Time: 3

Priority: 2

Process 3

Arrival Time: 2

Burst Time: 2

Priority: 1

Process 4

Arrival Time: 3

Burst Time: 2

Priority: 4

PID	AT	BT	PR	WT	TAT	CT
1	0	2	3	4	6	1
2	1	3	2	2	5	5
3	2	2	1	0	2	4
4	3	2	4	4	6	9

Grant Chart

P2 | P3 | P2 | P1 | P4 |
0 2 4 5 7 9

Average Waiting Time: 2.6 ms

Average Turnaround Time: 4.75 ms

* Implementing Round Robin (Experiment with different quantum sizes for RR algorithm).

```
#include <stdio.h>
```

```
struct Process
```

```
{
    int pid, at, bt, ct, tot, wt, rt;
```

```
};
```

```
int main () {
```

```
    int n, tq;
```

```
    struct process p[20];
```

```
    int queue[100];
```

```
    int visited[20] = {0};
```

```
    int front = 0, rear = 0;
```

```
    int time, completed = 0;
```

```
    int order[200];
```

```
    int timeMark[20];
```

```
    int k = 0;
```

```
    float avgwt = 0, avgat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].pid = i+1;
```

```
        printf("Enter Process %d\n", i+1);
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &p[i].at);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &p[i].bt);
```

```
        p[i].rt = p[i].bt;
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = i+1; j < n; j++) {
```

```
            if (p[i].at > p[j].at) {
```

```
                struct Process temp = p[i];
```

```
                p[i] = p[j];
```

```
                p[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("Enter Time Quantum: ");
```

```
    scanf("%d", &tq);
```

```
    time = p[0].at;
```

```
    timeMark[0] = time;
```

```
    while (completed < n) {
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (p[i].at <= time && visited[i] == 0) {
```

```
                queue[rear++] = i;
```

```
                visited[i] = 1;
```

```
            }
```

```
        }
```

```
        if (front == rear) {
```

```
            time++;
```

```
            continue;
```

```
        }
```



```
int idx = queue.front++;
order[k] = p[idx].pid;
```

```
if (p[idx].wt > tq) {
    p[idx].wt = tq;
    time += tq;
```

5

elaz

```
time += p[idx].wt;
```

$p[idx].st = 0;$

$p[id \times 7].ct = time;$

$$p[idx].tot = p[idx].cl - p[idx].at.$$

$p[id_x].wt = p[id_x].tat - p[id_x].bf;$

$$\text{avgwt} += p[\text{idx}].\text{wt};$$
$$\text{avgval} += \text{p[idex]}.val;$$

completed ++;

2

 $K + 4j$

timeMark[k] = time;

```
for(int i=0; i<n; i++) {
```

if $p[i]$ at $\leq$ time ss visited $[i] = -1$

```
queue[rear+1] = i;
```

visited[i] = 1;

3

3

if (p[id_x].x > 0) queue[current++] = id_x;

```

for (int i=0; i<n-1; i++) {
    for (int j=i+1; j<n; j++) {
        if (p[i].pid > p[j].pid) {
            struct process temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

```

9

3

```
printf("m - Round Robin scheduling - m");  
printf("m Grantt chart: m");
```

if (timeMark[0] != 0) printf ("1 %s", "IDLE");

```
for(int i=0; i<k; i++)
```

```
char temp[10];
```

```
printf("temp, %d", order[i]);
```

pointf ("1 %-5s", temp);

4

pointy ("\\n"),

```
if (timeMark[0] != 0) printf("%d", 0);
```

```
for (int i=0; i<n; i++)
```

```
printf("P%d lt %d lt %d lt %d\n",
```

pcij.pid, pcij.at, pcij.bt, pcij.ct,

```
priz.tal, priz.wat);
```

2

point ("In Average TAT = %2", avg tat/n);

pointy (1" in Average WT = $\frac{1}{2}$ in, avg wt/n);

5. detikha;

Output:

Enter number of processes: 3

Process 1

Arrival Time: 2

Burst Time: 3

Process 2

Arrival Time: 1

Burst Time: 2

Process 3

Arrival Time: 4

Burst Time: 2

Enter Time Quantum: 2

-Round Robin scheduling-

Gantt chart:

| IDLE | P2 | P1 | P3 | P1 |
0 1 3 5 7 8

Process	AT	BT	CT	TAT	WT
P1	2	3	8	6	3
P2	1	2	3	2	0
P3	4	2	7	3	1

Average TAT = 3.67

Average WT = 1.33

Output:

Enter number of Processes: 3

Process 1

Arrival Time: 2

Burst Time: 3

Process 2

Arrival Time: 1

Burst Time: 2

Process 3

Arrival Time: 4

Burst Time: 2

Enter Time Quantum: 3

-Round Robin scheduling-

Gantt chart:

| IDLE | P2 | P1 | P3 |
0 1 3 6 8

Process	AT	BT	CT	TAT	WT
P1	2	3	6	4	1
P2	1	2	3	2	0
P3	4	2	8	4	2

Average TAT = 2.33

Average WT = 1.00

Comparison:

when Time Quantum increases, Avg TAT and WT decrease.

* Implementing multi-level queue scheduling

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
struct Process {
```

```
    int pid;
    char type[10];
    int arrivalTime;
    int burstTime;
    int remainingTime;
    int completionTime;
    int waitingTime;
    int turnaroundTime;
```

```
};
```

```
struct Gantt {
```

```
    int pid;
    int start;
    int end;
```

```
};
```

```
int mfscheduling (struct Process queue[], int n, int  
startTime, struct Gantt gant[], int *gcount)
```

```
{
    int currentTime = startTime;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (currentTime < queue[i].arrivalTime) {
```

```
            currentTime = queue[i].arrivalTime;
```

```
        }
```

```
        gant[*gcount].pid = queue[i].pid;
        gant[*gcount].start = currentTime;
        gant[*gcount].end = currentTime + queue[i].burstTime;
        (*gcount)++;
```

```
        queue[i].completionTime = gant[*gcount-1].end;
        queue[i].turnaroundTime = queue[i].completionTime - queue[i].arrivalTime;
        queue[i].waitingTime = queue[i].turnaroundTime - queue[i].burstTime;
```

```
        currentTime = queue[i].completionTime;
```

```
    }
```

```
    return currentTime;
```

```
void roundRobinScheduling (struct Process queue[], int n,  
int start, int current Time = start Time;  
int completed = 0;
```

```
{
    for (int i = 0; i < n; i++) {
```

```
        queue[i].remainingTime = queue[i].burstTime;
```

```
    }
```

```
    while (completed < 0) {
```

```
        int done = 1;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (queue[i].remainingTime > 0 && queue[i].arrivalTime <= currentTime) {
```

```
                done = 0;
```

```
                gant[*gcount].pid = queue[i].pid;
```

```
                gant[*gcount].start = currentTime;
```



```

if (queue[i].remainingTime > quantum) {
    currentTime += quantum;
    queue[i].remainingTime -= quantum;
} else {
    currentTime += queue[i].remainingTime;
    queue[i].completionTime = currentTime;
    queue[i].turnaroundTime = queue[i].completionTime -
        queue[i].arrivalTime;
    queue[i].waitingTime = queue[i].turnaroundTime -
        queue[i].burstTime;
    queue[i].remainingTime = 0;
    completed++;
}

```

```

gantt[*gcount].end = currentTime;
(*gcount)++;
}

```

```

if (!done) break;
}

```

```

void printTable(struct Process processes[], int n) {

```

```

    double totalWT = 0, totalTAT = 0;
    printf("\nID\tType\tAT\tCT\tWT\tTAT\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            processes[i].pid, processes[i].type, processes[i].arrivalTime,
            processes[i].burstTime, processes[i].completionTime,
            processes[i].waitingTime, processes[i].turnaroundTime);
        totalWT += processes[i].waitingTime;
        totalTAT += processes[i].turnaroundTime;
    }
}

```

```

printf("\nAverage Waiting Time: %.2f, totalTAT: %d\n",
    printf("\nAverage Turnaround Time: %.2f, totalTAT: %d\n",

```

```

void printGantt (struct Gantt gantt[], int gcount) {
    printf("\n -- Gantt Chart -- \n");
    for (int i = 0; i < gcount; i++) {
        printf("----");
    }
}

```

```

printf("\n");
for (int i = 0; i < gcount; i++) {
    printf("P%d", gantt[i].pid);
}

```

```

printf("\n");
for (int i = 0; i < gcount; i++) {
    printf("----");
}

```

```

printf("\n");
printf("%d", gantt[0].start);
for (int i = 0; i < gcount; i++) {
    printf(" %d", gantt[i].end);
}

```

```

printf("\n");
}

```

```

int main() {

```

```

    int n, quantum;
    printf("Enter no. of process: ");
    scanf("%d", &n);

```

```

    struct Process processes[n];

```




```
struct process systemQueue[n], userQueue[n];
int syscount = 0, usercount = 0;
```

```
for (int i=0; i<n; i++) {
    processes[i].pid = i;
    printf("In Process %m", i);
    printf("Enter arrival time: ");
    scanf("%d", &processes[i].arrivalTime);
    printf("Enter burst time: ");
    scanf("%d", &processes[i].burstTime);
    printf("Enter type (0 = System, 1 = User): ");
    int t;
    scanf("%d", &t);
    if (t==0) strcpy(processes[i].type, "System");
    else strcpy(processes[i].type, "User");
}
```

```
if (t==0) systemQueue[syscount++] = processes[i];
else userQueue[usercount++] = processes[i];
```

```
printf("In Enter time quantum for Round Robin: ");
scanf("%d", &quantum);
```

```
for (int i=0; i<syscount; i++) {
    for (int j=i+1; j<syscount; j++) {
        if (systemQueue[i].arrivalTime < systemQueue[j].
            arrivalTime) {
            struct process temp = systemQueue[i];
            systemQueue[i] = systemQueue[j];
            systemQueue[j] = temp;
        }
    }
}
```



```
for (int i=0; i<syscount-1; i++) {
    for (int j=i+1; j<syscount; j++) {
        if (systemQueue[i].arrivalTime > systemQueue[j].
            arrivalTime) {
```

```
            struct process temp = systemQueue[i];
            systemQueue[i] = systemQueue[j];
            systemQueue[j] = temp;
        }
    }
}
```

```
struct Grant { grant[100];
int gcount = 0;
```

```
int lastCompletion = bfscheduling(systemQueue, syscount, 0,
    grant, gcount);
```

```
roundRobinscheduling(userQueue, usercount, lastCompletion,
    quantum, grant, gcount);
```

```
for (int i=0; i<syscount; i++) process(systemQueue[i].pid)
    = systemQueue[i];
```

```
for (int i=0; i<usercount; i++) process(userQueue[i].pid)
    = userQueue[i];
```

```
printTable(processes, n);
printGrant(grant, gcount);
return 0;
```


Output:

Enter no. of processes: 4

Process 0

Enter arrival time: 0

Enter Burst time: 5

Enter type (0=System, 1=User): 0

Process 1

Enter arrival time: 2

Enter Burst time: 3

Enter type (0=System, 1=User): 1

Process 2

Enter arrival time: 1

Enter Burst time: 4

Enter type (0=System, 1=User): 0

Process 3

Enter arrival time: 3

Enter Burst time: 2

Enter type (0=System, 1=User): 1

Enter time quantum for Round Robin: 3.

ID	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
1	User	2	3	12	7	10
2	System	1	4	3	4	8
3	User	3	2	14	9	11

Average waiting time: 5.00
Average Turnaround time: 8.5

Gantt Chart

P0	P2	P1	P3
0 3	3 7	12 14	11

Q. 2

- * Write a C program to simulate Real-Time CPU scheduling algorithms:
- Rate Monotonic
 - Earliest-deadline first

```
#include <stdio.h>
#include <math.h>
```

```
typedef struct {
```

```
    int id, burst, deadline, period, completions, waiting, turnaround;
```

```
    process;
```

```
void printTableEDF (Process p[], int n) {
    printf("== Earliest Deadline First == \n");
    printf("ID\tBT\tDeadline\tCT\tWT\tAT\n");
    for (int i=0; i<n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].id, p[i].burst, p[i].deadline,
            p[i].completion, p[i].waiting, p[i].turnaround);
    }
}
```

```
void printTableRMS (Process p[], int n) {
    printf("== Rate Monotonic Scheduling == \n");
    printf("ID\tBT\tPeriod\tCT\tWT\tAT\n");
    for (int i=0; i<n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
        p[i].id, p[i].burst, p[i].period,
        p[i].completion, p[i].waiting, p[i].turnaround);
    }
}

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    process p[n], edf[n], rms[n];

    for (int i=0; i<n; i++) {
        printf("Enter Burst Time, Deadline, Period for Process %d:", i);
        scanf("%d %d %d", &p[i].burst, &p[i].deadline, &p[i].period);
        p[i].id = i;
    }

    for (int i=0; i<n; i++) {
        edf[i] = p[i];
        rms[i] = p[i];
    }

    for (int i=0; i<n-1; i++) {
        for (int j=i+1; j<n; j++) {
            if (edf[i].deadline > edf[j].deadline) {
                Process temp = edf[i];
                edf[i] = edf[j];
                edf[j] = temp;
            }
        }
    }
}
```



```

for (int i=0; i<n; i++) {
    for (int j=i+1; j<n; j++) {
        if (oms[i].period > oms[j].period) {
            process temp = oms[i];
            oms[i] = oms[j];
            oms[j] = temp;
        }
    }
}

int time = 0;
for (int i=0; i<n; i++) {
    time += edf[i].burst;
    edf[i].completion = time;
    edf[i].turnaround = edf[i].completion;
    edf[i].waiting = edf[i].turnaround - edf[i].burst;
}

```

```

time = 0;
for (int i=0; i<n; i++) {
    time += oms[i].burst;
    oms[i].completion = time;
    oms[i].turnaround = oms[i].completion;
    oms[i].waiting = oms[i].turnaround - oms[i].burst;
}

```

```

double U = 0.0;
for (int i=0; i<n; i++) {
    U += (double) p[i].burst / p[i].period;
}

double rmsBound = (pow(U, 0.5) * (1.0/n) * 1.0);

```

```

printTableEDF (edf, n);
if (U <= 1.0)
    printf ("Schedulability (EDF): Task set is schedulable (U=%2f <= 1) \n", U);
else
    printf ("Schedulability (EDF): Task set NOT schedulable (U=%2f > 1) \n", U);

```

```

PrintGrantChart (edf, n, "EDF");
printTableRMS (oms, n);
if (U <= rmsBound)
    printf ("Schedulability (RMS): Task set is schedulable (U=%2f <= Bound=%2f) \n", U, rmsBound);
else
    printf ("Schedulability (RMS): Task set NOT schedulable (U=%2f > Bound=%2f) \n", U, rmsBound);

```

```

return 0;

```

return 0;

Output 1:

Enter number of processes: 3

Enter process details

Enter Burst Time, Deadline, Period for process 0: 3 7 20

Enter Burst Time, Deadline, Period for process 1: 2 4 5

Enter Burst Time, Deadline, Period for process 2: 2 8 10

= = Earliest Deadline First E = =

ID	BT	Deadline	CT	WT	TAT
1	2	4	2	0	2
0	3	7	5	2	5
2	2	8	7	5	7

Schedulability (EDF): Task set is schedulable ($U=0.75 \leq 1$)

EDF Gantt Chart

1	2	1	3	2	IDLE	2	3	IDLE	2	IDLE
0	2	5	7	9	10	12	14	15	17	20

= = RMS = =

ID	BT	Period	CT	WT	TAT
1	2	5	2	0	2
2	2	10	4	2	4
0	3	20	7	4	7

Schedulability (RMS): Task set is schedulable ($U=0.75 \leq 0.79$)

$\leq$ Bound
= 0.79

RMS Gantt Chart

1	2	1	2	2	IDLE	2	3	IDLE	2	IDLE	
0	2	4	5	7	9	10	12	14	15	17	20

Output 2:

Enter number of processes: 3

Process 0

Enter Burst Time, Deadline, Period for process: 2 5 5

Enter Burst Time, Deadline, Period for process: 2 7 7

Enter Burst Time, Deadline, Period for process: 3 8 8

= = EDF scheduling = =

CPU Utilization: 1.06

Not schedulable

= = RMS = =

CPU Utilization: 1.06

Rm Bound: 0.7798

Not schedulable

$\frac{P}{21/2/24}$

Q. Consider two processes A and B. Two semaphore variables S and T are used to synchronize the processes. S is initialized to 0 and T is initialized to 1. Processes are scheduled in following order: A B A B B A A B A
What will be printed on the screen?

Process A:

```
while (1) {
```

```
    wait(S);
```

```
    printf('P');
```

```
    printf('P');
```

```
    signal(T);
```

```
}
```

Process B:

```
while (1) {
```

```
    wait(T);
```

```
    printf('I');
```

```
    printf('I');
```

```
    signal(S);
```

```
}
```

Initial	S = 0	T = 1	Remark
A	0	1	Process A is Blocked(wait)
B	1	0	11
A	0	1	11pp
B	1	0	11pp11
B	1	0	process B is Blocked
A	0	1	11pp11pp
A	0	1	process A is Blocked
B	1	0	11pp11pp11
A	0	1	11pp11pp11pp

★ Producer Consumer:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#define SIZE 3
```

```
#define MAX 10
```

```
int buffer[SIZE];
```

```
int in = 0, out = 0;
```

```
sem_t empty, full, mutex;
```

```
void production (void *arg) {
```

```
    for (int i = 0; i < MAX; i++) {
```

```
        sem_wait (&empty);
```

```
        sem_wait (&mutex);
```

```
        buffer[in] = i;
```

```
        printf ("Produced: %d at buffer [%d]\n", i, in);
```

```
        in = (in + 1) % SIZE;
```

```
        sem_post (&mutex);
```

```
        sem_post (&full);
```

```
}
```

```
return NULL;
```

```
}
```

```
void consumer (void *arg) {
```

```
    for (int i = 0; i < MAX; i++) {
```

```
        sem_wait (&full);
```

```
        sem_wait (&mutex);
```

```
        int item = buffer[out];
```

```
        printf ("Consumed: %d from buffer [%d]\n",
```

```
            item, out);
```

```
        out = (out + 1) % SIZE;
```

```
        sem_post (&mutex);
```

```
        sem_post (&empty);
```

```
}
```


return NULL;

```

int main() {
    pthread_t p, c;
    sem_init(&empty, 0, 0);
    sem_init(&full, 0, 1);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);
    return 0;
}

```

Output:

Sum

```

Produced: 0 at buffer [0]
Consumed: 0 from buffer [0]
Produced: 1 at buffer [1]
Consumed: 1 from buffer [1]
Produced: 2 at buffer [2]
Consumed: 2 from buffer [2]
Produced: 3 at buffer [0]
Produced: 4 at buffer [1]
Consumed: 3 from buffer [0]
Produced: 5 at buffer [2]
Consumed: 4 from buffer [1]
Produced: 6 at buffer [0]
Produced: 7 at buffer [1]
Consumed: 5 from buffer [2]
Produced: 8 at buffer [2]
Consumed: 6 from buffer [0]
Produced: 9 at buffer [0]
Consumed: 7 from buffer [1]
Consumed: 8 from buffer [2]
Consumed: 9 from buffer [0]

```


Trace:

show sum values

Step	Action	Printed op	in	out	buffer
1	P	Produced 0 at buffer[0]	1	0	0
2	C	Consumed 0 from buffer[0]	1	1	0
3	P	Produced 1 at buffer[1]	2	1	1
4	C	Consumed 1 from buffer[1]	2	2	1
5	P	Produced 2 at buffer[2]	0	2	2
6	C	Consumed 2 from buffer[2]	0	0	2
7	P	Produced 3 at buffer[0]	1	0	0
8	P	Produced 4 at buffer[1]	2	0	1
9	C	Consumed 3 from buffer[0]	2	1	0
10	P	Produced 5 at buffer[2]	0	1	2
11	C	Consumed 4 from buffer[1]	0	2	1
12	P	Produced 6 at buffer[0]	1	2	0
13	P	Produced 7 at buffer[1]	2	2	1
14	C	Consumed 5 from buffer[2]	2	0	2
15	P	Produced 8 at buffer[0]	0	0	2

Step	Op	Buffer	in	out	empty	full	mutex
1	Producer inserts	[A--]	1	0	2	1	1
2	Consumer inserts	[AB--]	2	0	1	2	1
3	Producer inserts	[ABC]	0	0	0	3	1
4	Consumer takes	[A,BC]	0	0	0	3	1
5	Consumer removes	[-,BC]	0	1	1	2	1
6	Producer inserts	[D,BC]	1	1	0	3	1
7	Consumer removes	[D,-,C]	1	2	1	2	1

Dining-Philosopher's Problem

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#define N 5
#define ROUNDS
sem_t forks[N];
sem_t room;
```

```
void * philosopher (void * arg) {
    int id = *(int *) arg;
    int left = id;
    int right = (id+1)%N;
    for (int i=0; i<ROUNDS; i++) {
        printf("Philosopher %d is thinking", id);
        usleep(200000);
        sem_wait(&room);
        sem_wait(&forks[left]);
        printf("Philosopher %d picked up left
            fork %d, id+1, left);
        sem_wait(&forks[right]);
        printf("Philosopher %d picked up right
            fork %d, id+1, right);

        printf("Philosopher %d is eating", id);
        usleep(200000);
        sem_post(&forks[left]);
        sem_post(&forks[right]);
        printf("Philosopher %d put down forks %d,
            and %d in", id+1, left, right);
    }
}
```



```
sem_t room (&room);  
while (150000);
```

```
room = NULL;
```

```
int main() {
```

```
pthread_t ph[N];
```

```
int id[N];
```

```
sem_init (&room, 0, N-1);
```

```
for (int i=0; i<N; i++) {  
    id[i] = i;
```

```
pthread_create (&ph[i], NULL, philosopher, &id[i]);
```

```
for (int i=0; i<N; i++) {
```

```
pthread_join (&ph[i], NULL);
```

```
for (int i=0; i<N; i++) {
```

```
sem_destroy (&forks[i]);
```

```
sem_destroy (&room);
```

```
return 0;
```

```
}
```

C/P

Philosopher 1 is thinking

Philosopher 2 is thinking

Philosopher 3 is thinking

Philosopher 4 is thinking

Philosopher 5 is thinking

Philosopher 4 picked up left fork 3

Philosopher 5 picked up left fork 4

Philosopher 5 picked up right fork 0

Philosopher 5 is eating

Philosopher 2 picked up left fork 1

Philosopher 3 picked up left fork 2

Philosopher 5 picked up down fork 4 and 0

Philosopher 4 is eating

Philosopher 51 is picked up left fork 0

20/1/20

Implementing Safety Algorithm:

Include <stdio.h>

int main() {

int n, m, process;

printf("Enter no. of processes: ");

scanf("%d", &n);

printf("Enter no. of resource types: ");

scanf("%d", &m);

int alloc[m][m], max[m][m], need[m][m];

int avail[m];

printf("Enter Allocation Matrix: \n");

for (int i = 0; i < n; i++) {

for (int j = 0; j < m; j++) {

scanf("%d", &alloc[i][j]);

}

printf("Enter Max Matrix: \n");

for (int i = 0; i < n; i++) {

for (int j = 0; j < m; j++) {

scanf("%d", &max[i][j]);

}

printf("Enter Available Resources: \n");

for (int j = 0; j < m; j++) {

scanf("%d", &avail[j]);

}

for (int i = 0; i < n; i++) {

for (int j = 0; j < m; j++) {

need[i][j] = max[i][j] - alloc[i][j];

}

printf("Enter Allocation Matrix: \n");

for (int i = 0; i < n; i++) {

for (int j = 0; j < m; j++) {

scanf("%d", &alloc[i][j]);

printf("Enter \n"); } printf("Enter process no. making req.:\n");

scanf("%d", &process);

int finish[n], safeSeq[n];

for (int i = 0; i < n; i++)

finish[i] = 0;

int work[m];

for (int i = 0; i < m; i++)

work[i] = avail[i];

int count = 0;

while (count < n) {

int found = 0;

for (int i = 0; i < n; i++) {

if (finish[i] == 0) {

int j;

for (j = 0; j < m; j++) {

if (need[i][j] > work[j])

break;

}

if (j == m) {

for (int k = 0; k < m; k++) {

work[k] += alloc[i][k];

}

safeSeq[count++] = i;

finish[i] = 1;

found = 1;

}


```
if (found == 0) {
    printf("System is not in safe state!\n");
    return 0;
}
```

```
printf("System is in safe state. In safe sequence:");
for (int i = 0; i < m; i++) {
    printf("P%d ", seq[i]);
    if (i % 10 == 0) printf("\n");
}
printf("\n");
return 0;
```

Output:

Enter no. of processes: 5

Enter no. of resource types: 3

Enter allocation matrix:

0 1 2
2 0 0
3 0 2
2 1 1
0 0 2

Enter max matrix:

7 5 3
3 2 0
3 0 2
2 2 2
4 3 3

Enter Available Resources:

3 3 2

Need Matrix:

7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in safe state.

Safe Sequence: P1 → P3 → P4 → P0 → P2

Request

```
void requestResources (int process, int request[]) {
```

```
    for (int i = 0; i < m; i++) {
```

```
        if (request[i] > need[process][i]) {
            printf("Error: Request exceeds need\n");
            return;
        }
    }
```

```
    for (int i = 0; i < m; i++) {
```

```
        if (request[i] > need[avail][i]) {
            printf("Error: Request exceeds need\n");
        }
    }
```



```

int requestResource (int n, int m, int process,
                    int alloc[n][m], int need[n][m],
                    int avail[m]) {

    int request[m];

    printf("Enter req. vector for process P-%d: \n", process);
    for (int i = 0; i < m; i++) {
        scanf("%d", &request[i]);
    }

    for (int i = 0; i < m; i++) {
        if (request[i] > need[process][i]) {
            printf("Error: Req. exceeds need!\n");
            return 0;
        }
    }

    for (int i = 0; i < m; i++) {
        if (request[i] > avail[i]) {
            printf("Resources not available;\n");
            printf("Process must wait");
            return 0;
        }
    }

    for (int i = 0; i < m; i++) {
        avail[i] = request[i];
        alloc[process][i] += request[i];
        need[process][i] -= request[i];
    }

    printf("Request allocated temporarily. checking safety");
    return 1;
}

```

Output:

Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:

0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix:

7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:

3 3 2

Enter process number making request: 1

Enter request vector for process P1:

1 0 2

Request allocated temporarily. checking safety

System is in safe state

Request Granted

Safe Sequence: P1 → P3 → P4 → P0 → P2

* Deadlock Detection:

```
#include <stdio.h>
#define MAX 10
```

```
int main() {
    int n, m;
    int alloc[MAX][MAX], request[MAX][MAX];
    int avail[MAX];
    int finish[MAX];
    int i, j, count = 0;
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter number of resource types: ");
scanf("%d", &m);
```

```
printf("\nEnter Allocation Matrix:");
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++) {
        scanf("%d", &alloc[i][j]);
    }
}
```

```
printf("Enter Request Matrix: ");
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++) {
        scanf("%d", &request[i][j]);
    }
}
```

```
printf("Enter Available Resources:");
for (j = 0; j < m; j++) {
    scanf("%d", &avail[j]);
}
```

```
for (i = 0; i < n; i++) {
    int flag = 0;
    for (j = 0; j < m; j++) {
        if (alloc[i][j] != 0) {
            flag = 1;
            break;
        }
    }
```

```
if (flag == 0) finish[i] = 1;
```

```
else finish[i] = 0;
```

```
while (count < n) {
```

```
    int found = 0;
```

```
    for (i = 0; i < n; i++) {
```

```
        if (finish[i] == 0) {
```

```
            int possible = 1;
```

```
            for (j = 0; j < m; j++) {
```

```
                if (request[i][j] > avail[j]) {
```

```
                    possible = 0;
```

```
                    break;
```

```
            if (possible) {
```

```
                for (j = 0; j < m; j++) {
```

```
                    avail[j] += alloc[i][j];
```




```
finish[i] = 1;  
found = 1;  
count++;
```

```
}
```

```
}
```

```
if (found == 0)  
    break;
```

```
}
```

```
int deadlock = 0;
```

```
printf("In Processes in Deadlock : ");
```

```
for (i = 0; i < n; i++)
```

```
    if (finish[i] == 0)
```

```
        printf("P%d", i);
```

```
        deadlock = 1;
```

```
}
```

```
}
```

```
if (deadlock == 0)
```

```
    printf("None (No Deadlock)");
```

```
printf("\n");
```

```
return 0;
```

Output:

Enter number of processes : 4

Enter number of resource types : 3

Enter Allocation matrix:

1 0 2

2 1 1

1 0 3

1 2 2

Enter Request matrix:

0 0 1

1 0 2

0 0 0

3 3 0

Enter Available Resources:

0 0 0

Processes in Deadlock : P3

Q. Given a free memory partition 400K, 700K, 200K, 300K & 600K. How would each of first fit, best fit and worst fit algorithms work to place process of 212K, 612K, 312K, 526K (in order)

Fit

212K	512K			312K
------	------	--	--	------

4500K → 7000K → 200K → 200K → 200K →

526K cannot allotted with memory

RAY F:4

312K	526K	212K	612K
------	------	------	------

$$K_{400K} \rightarrow 200K \rightarrow 200K \rightarrow 300K \rightarrow 600K \rightarrow$$

Word + Fit

32K	212K			50K
-----	------	--	--	-----

1000K * 1000K * 200K * 300K * 600K *

526K cannot be allocated with memory

* Write a C program to simulate the following contiguous memory allocation techniques:

- a) Worst-fit
- b) Best-fit
- c) First-fit

```
#include <stdio.h>
```

```
void yjzotFit (int blockSize[], int blocks, int processSize[],
```

```
int allocation[processes];
for(int i=0; i<processes; i++) allocation[i] = -1;
for(int i=0; i<processes; i++) {
    for(int j=0; j<blocks; j++) {
        if(blockSize[j] >= processSize[i]) {
            allocation[i] = j;
            blockSize[j] -= processSize[i];
            break;
        }
    }
}
```

```
printf("m FIT Allocation: \n");
printf("Process No \t process Size \t Block No \n");
for (int i=0; i<processes; i++)
    printf("%d \t %d \t %d", i+1, processSize[i]);
    if (allocation[i] != -1) printf("%d \n", allocation[i]);
    else printf("Not Allocated \n");
```

```
void bestFit (int blockSize[], int blocks, int processSize[], int processes,
             int allocation[processes]) {
    for (int i = 0; i < processes; i++) allocation[i] = -1;
```




```

for (int i=0; i<processes; i++) {
    int bestIdx = -1;
    for (int j=0; j<blocks; j++) {
        if (blockSize[j] >= processSize[i]) {
            if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                bestIdx = j;
        }
    }
}

```

```

if (bestIdx != -1) {
    allocation[i] = bestIdx;
    blockSize[bestIdx] -= processSize[i];
}

```

```

printf("In BEST FIT Allocation: \n");
printf("Process No\tProcess Size\tBlock No\n");
for (int i=0; i<processes; i++) {
    printf("%d\t%d\t%d", i+1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i]+1);
    else
        printf("Not Allocated\n");
}

```

```

void worstFit(int blockSize[], int blocks, int processSize[],
              int processes) {
    int allocation[processes];
    for (int i=0; i<processes; i++) allocation[i] = -1;
    for (int i=0; i<processes; i++) {
        int worstIdx = -1;
        for (int j=0; j<blocks; j++) {

```



```

        if (blockSize[j] >= processSize[i]) {
            if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                worstIdx = j;
        }
    }
    if (worstIdx != -1) {
        allocation[i] = worstIdx;
        blockSize[worstIdx] -= processSize[i];
    }
}

```

```

printf("In Worst FIT Allocation: \n");
printf("Process No\tProcess Size\tBlock No\n");
for (int i=0; i<processes; i++) {
    printf("%d\t%d\t%d", i+1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i]+1);
    else
        printf("Not Allocated\n");
}

```

```

int main() {

```

```

    int blocks, processes;
    printf("Enter number of memory blocks: ");
    scanf("%d", &blocks);
    int blockSize[blocks];
    printf("Enter sizes of memory blocks: \n");
    for (int i=0; i<blocks; i++)
        scanf("%d", &processes);
    int processSize[processes];
    printf("Enter sizes of processes: \n");
    return 0;
}

```



```
for (int i=0; i<processes; i++)
```

Output:

Enter number of memory blocks: 6

Enter size of each memory block:

300 600 350 200 700 125

Enter number of process: 5

Enter size of process:

115 500 358 200 375

FIRST FIT

Process No.	Process Size	Block No
1	115	1
2	500	2
3	358	5
4	200	3
5	375	Not allocated

BEST FIT

Process No	Process Size	Block No
1	115	6
2	500	2
3	358	5
4	200	4
5	375	Not allocated

WORST FIT

Process No	Process Size	Block No
1	115	5
2	500	2
3	358	Not allocated
4	200	3
5	375	Not allocated

Write a C program to simulate page replacement algo:
i) FIFO ii) LRU iii) Optimal

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
void fifo (int pages[], int n, int num-pages) {
    int frames[num-pages];
    for (int i=0; i<num-pages; i++) frames[i] = -1;
    int page-faults = 0, next = 0;
    printf("\n FIFO page replacement : \n");
    for (int i=0; i<n; i++) {
        int page = pages[i], found = 0;
        for (int j=0; j<num-pages; j++) {
            if (frames[j] == page) { found = 1; break; }
        }
        if (!found) {
            page-faults++;
            frames[next] = page;
            next = (next+1) % num-pages;
        }
        printf("Page %d", page);
        for (int j=0; j<num-pages; j++) printf("%d", frames[j]);
        printf("\n");
    }
    printf("Total page faults (FIFO): %d \n", page-faults);
}
```

```
void optimal (int page[], int n, int num-pages) {
    int frames[num-pages];
    for (int i=0; i<num-pages; i++) frames[i] = -1;
    int page-faults = 0;
    printf("\n Optimal page Replacement : \n");
    for (int i=0; i<n; i++) {
```



```

int page = pages[i], found = 0;
for (int j = 0; j < num-frames; j++) {
    if (frames[j] == page) {
        found = 1;
        break;
    }
    if (!found) {
        page-faults++;
        int placed = 0;
        for (int j = 0; j < num-frames; j++) {
            if (frames[j] == -1) {
                frames[j] = page;
                placed = 1;
                break;
            }

```

```

        }
        if (!placed) {
            int farthest = -1, replace-index = -1;
            for (int j = 0; j < num-frames; j++) {
                int next-use = INT_MAX;
                for (int k = i+1; k < n; k++) {
                    if (pages[k] == frames[j]) {
                        next-use = k;
                        break;
                    }
                    if (next-use > farthest) {
                        farthest = next-use;
                        replace-index = j;
                    }
                }
            }

```

```

            frames[replace-index] = page;
        }
    }
}

```

```

printf("Page %d", page);
for (int j = 0; j < num-frames; j++)
    printf("%d", frames[j]);
printf("\n");
}
printf("Total Page Faults (Optimal): %d\n", page-faults);

```

```

void LRUinit (pages[], int n, int num-frames) {
    int frames[num-frames], last-used[num-frames];
    for (int i = 0; i < num-frames; i++) {
        frames[i] = -1;
        last-used[i] = -1;
    }
}

```

```

int page-faults = 0;
printf("In LRU Page Replacement\n");
for (int i = 0; i < num-frames; i++) {
    int page = pages[i], found = 0;
    for (int j = 0; j < num-frames; j++) {
        if (frames[j] == page) {
            found = 1;
            last-used[j] = i;
            break;
        }
    }
    if (!found) {
        page-faults++;
        int placed = 0;
        for (int j = 0; j < num-frames; j++) {
            if (frames[j] == -1) {
                frames[j] = page;
                last-used[j] = i;
                placed = 1;
                break;
            }
        }
    }
}

```

```

if (!found) {
    page-faults++;
    int placed = 0;
    for (int j = 0; j < num-frames; j++) {
        if (frames[j] == -1) {
            frames[j] = page;
            last-used[j] = i;
            placed = 1;
            break;
        }
    }
}
}

```



```

if (!placed) {
    int dou_index = 0;
    for (int j = 0; j < num_frames; j++) {
        if (last_used[j] < last_used[dou_index])
            dou_index = j;
    }
    frames[dou_index] = page;
    last_used[dou_index] = i;
}

printf("Page %d", page);
for (int j = 0; j < num_frames; j++) {
    printf("%d", frames[j]);
}
printf("\n");

printf("Total page faults (LRU) %d \n", page);

int main() {
    int num_frames, n;
    printf("Enter number of frames:");
    scanf("%d", &num_frames);
    printf("Enter length of reference string");
    scanf("%d", &n);
    int pages[n];
    printf("Enter the reference string");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    fifo(pages, n, num_frames);
    optimal(pages, n, num_frames);
    lru(pages, n, num_frames);
    return 0;
}

```

OUTPUT

Enter number of frames: 3

Enter length of reference string: 120

Enter the reference string (space-separated):

7 2 3 12 5 3 4 6 7 7 1 0 5 4 6 2 3 0 1

FIFO Page Replacement:

Page 7: 7 -1 -1

Page 2: 7 2 -1

Page 3: 7 2 3

Page 1: 1 2 3

Page 2: 1 2 3

Page 5: 1 5 3

Page 3: 1 5 3

Page 4: 1 5 4

Page 6: 6 5 4

Page 7: 6 7 4

Page 7: 6 7 4

Page 1: 6 7 1

Total page faults (FIFO): 9

Optimal Page Replacement:

Page 7: 7 -1 -1

Page 2: 7 2 -1

Page 3: 7 2 3

Page 1: 1 2 3

Page 2: 1 2 3

Page 5: 1 5 3

Page 3: 1 5 3

Page 4: 1 4 3

Page 6: 1 6 3

Page 7: 1 7 3

Page 7: 1 7 3

Page 1: 1 7 3

Total page faults (optimal): 8

LRU Page Replacement:

Page 7: 7 -1 -1

Page 2: 7 2 -1

Page 3: 7 2 3

Page 1: 1 2 3

Page 2: 1 2 3

Page 5: 1 2 5

Page 3: 3 2 5

Page 4: 3 4 5

Page 6: 3 4 6

Page 7: 7 4 6

Page 7: 7 4 6

Page 1: 7 1 6

Total page faults (LRU): 10